ElevenLabs — participant handout

BrainHack 2026 Workshop — Speech Recognition and Generation
Module 2 (55 min) · Instructors: Franciszek Kubala, Mateusz Poszelężny — KN AIMed AGH

> In this part we work mainly through the ElevenLabs website. You will upload your own recording, check the transcription, see how a conversation is split into speakers, and generate speech from text. You will also learn about voice cloning and how to get access via the API.

1. What will we do?

- **Speech-to-text (STT):** recording → text. In ElevenLabs this is done by the **Scribe** model.
- **Diarization:** splitting a conversation into utterances of individual speakers.
- **Text-to-speech (TTS):** text → synthetic speech in a chosen voice.
- **Voice cloning:** creating a voice from a sample, then using it to read new text.
- **API:** accessing the service from your own program, instead of clicking around the website manually.

Example: you record a conversation → you get text with speaker labels → you check for errors → you save the result.

2. What to prepare before the exercises?

- An ElevenLabs account, a browser, internet access and headphones.
- About **1 minute of your own, neutral speech** in a WAV or MP3 file. Speak clearly, in a quiet place, with no music or other people in the background.

Account and costs: the Free plan includes STT and TTS within the available limits. **Instant Voice Cloning is available from the Starter plan**, and Professional Voice Cloning from Creator. Current availability: ElevenLabs pricing.

3. Exercise A — transcribing your own voice (STT)

Step by step

1. Log in to ElevenLabs and open **Speech to Text**.
2. Click **Transcribe files** and select your own file.
3. Set the language (e.g. English or Polish) or leave **Detect**, i.e. automatic language detection.
4. The optional **Tag audio events** flag marks sounds such as laughter.
5. Click **Upload files** and wait for the result. Then open the transcript by selecting the file name.
6. Click any word to listen to the corresponding audio fragment. Fix the errors you notice in the editor.

What to check?

Listen to 15–20 seconds and compare the audio with the text. Pay attention to proper names, numbers and negations: an omitted "not" can reverse the meaning of an utterance. Also check whether the model skipped a fragment or added words.

4. Exercise B — transcribing and diarizing a conversation

Diarization answers the question "which speaker is talking at a given moment?". You get labels such as `Speaker 1` and `Speaker 2`. The labels alone do not give names or confirm anyone's identity.

Step by step

1. In **Speech to Text**, upload the conversation file as in Exercise A. On the website, speaker segmentation is already included in the result; in the API you enable it with the `diarize` parameter.
2. Open the result and find the speaker labels.
3. Listen to speaker changes. Check that each utterance was assigned to the right speaker and that the same person keeps one label.
4. In the **Speakers** section, using the edit button, you can rename them to "Person A" and "Person B". That is your own annotation after listening.

Feature description: diarization and speaker names in the interface, diarization parameters in the API.

Exercise outcome: check at least two speaker changes. Record text errors and speaker-assignment errors separately — correct words may be attributed to the wrong speaker. Overlapping voices and noise make both tasks harder.

---

5. Saving the transcript and speaker labels

In the open transcript, click **Export** and choose the format that fits your task:

- **TXT:** text for reading and further work.
- **SRT or VTT:** subtitles with display times.
- **JSON, if available in a given export:** data for a program, e.g. text, timing and speaker identifier. You also get this result via the API.

Save the result as `diarization`. **Open the downloaded file and check that it contains the timings and speaker labels you need**

---

6. Exercise C — generating speech (TTS)

1. Open **Text to Speech**.
2. Paste a short text, e.g. the example below.
3. Choose a voice available on your account that supports your language, e.g. **Marcin - Polish Narrator** for Polish.
4. Click **Generate speech**, listen to the result and download the audio.

> "Welcome to the BrainHack workshop. Today we will check how a computer recognizes speech and reads written text. Take a short break, then compare the two recordings."

Exercise outcome: you will save a short audio file and point out what in its pronunciation or pace suits your use case, e.g. recording instructions for study participants.

---

7. Demo — cloning your own voice

Voice cloning creates a voice that can then be used to generate an utterance from new text. For a short demo we use **Instant Voice Cloning**, with a sample of about 1–2 minutes of clean speech from one person. Professional Voice Cloning requires much more material and an additional preparation process.

This is an instructor demo, not a participant exercise. You may only clone your own voice or the voice of a person who has given consent.

---

8. How to get access via the API?

An **API key** authorizes a program to use your account and its limits, letting you automate speech-processing workflows.

Creating the key on the website

1. In the account panel open **Developers → API Keys**.
2. Choose to create a key and name it, e.g. `brainhack-workshop`.
3. Keep the key restrictions and enable only the permissions you need: **Speech to Text** and **Text to Speech**.
4. Copy the key.
5. After the exercises you can delete the key if you no longer need it.

> **Important:** never show your API key to others or share it publicly.

What does the program send and receive?

- **STT:** it sends the file and model choice to `POST /v1/speech-to-text`. It receives **JSON**, i.e. data including the text, word timings and — with diarization enabled — speaker identifiers.
- **TTS:** it sends the text, the voice identifier (`voice_id`) and the model to `POST /v1/text-to-speech/{voice_id}`. It receives an **audio file**.

Outcome of this part: you can find where the key is created and use it in an example program.

9. Privacy — what to check on your account?

Both through the website and through the API you send data to a cloud service. This involves recordings, texts, transcripts and voice samples.

Exercise: go to the profile menu → **Terms and privacy → Data use**. Find the **Improve the models for everyone** option. If you do not want to share new data for model improvement, turn it off and save your choice with **Update your choice**. According to the provider's instructions, the choice applies to new data; it does not automatically delete previously uploaded files.

Before using the tool outside the workshop, establish:

- **Purpose and consent:** do people know about the recording, the upload to the provider and possible voice cloning?
- **Access:** who will see the files and results?
- **Storage:** how long does the data stay with the provider and how do you delete it?
- **Processing location:** which region and terms apply to your specific account or contract?

Disabling training and deleting a file are separate actions. Do not assume zero retention on a regular account either, i.e. the files you share are not deleted automatically.

In module 4 we will compare this with running Whisper yourself. **Colab is also a cloud — Google's.** Running a model on your own computer and running it in Colab are different data-processing locations.

10. Glossary

- **Transcription:** the text version of a recording.
- **Diarization:** assigning fragments of a recording to individual speakers.
- **Timestamp:** information about when a word or fragment occurred.
- **Voice cloning:** creating a voice from a recording to generate new utterances.
- **API:** how a program communicates with a service.
- **API key:** a confidential access key tied to the account and its permissions.
- **`voice_id`:** the identifier of the chosen voice.
- **`model_id`:** the identifier of the model performing the task.
- **JSON:** a data format readable by programs.
- **Retention:** the period for which data is stored.
- **Training opt-out:** declining the use of your data for model improvement, under the provider's terms.

11. Useful sources

- ElevenLabs — website and login.
- ElevenLabs documentation.
- Pricing and feature availability.
- Getting started with the API.
- STT via API example.
- Privacy policy.

Appendix A — first API calls (optional)

You can do this appendix after the Colab introduction. **You do not need to copy or understand every line during the demo.**

In a separate Colab cell install the library: `%pip install requests`. For STT, upload your file to the current session and adjust its name in the code. The example assumes the file `my_voice.wav`; for MP3 change that name.

A1. STT with diarization — file → text and JSON

```
import json
from getpass import getpass
from pathlib import Path

import requests

api_key = getpass("Paste your ElevenLabs API key (hidden): ").strip()
audio_path = Path("my_voice.wav")

with audio_path.open("rb") as audio_file:
    response = requests.post(
        "https://api.elevenlabs.io/v1/speech-to-text",
        headers={"xi-api-key": api_key},
        files={"file": (audio_path.name, audio_file)},
        data={
            "model_id": "scribe_v2",
            "language_code": "pol",
            "diarize": "true",
            "tag_audio_events": "false",
        },
        timeout=120,
    )

response.raise_for_status()
transcript = response.json()
print(transcript["text"])

Path("transcript.txt").write_text(transcript["text"], encoding="utf-8")
Path("transcript.json").write_text(
    json.dumps(transcript, ensure_ascii=False, indent=2), encoding="utf-8"
)

for word in transcript.get("words", []):
    if word.get("type") == "word":
        print(word.get("speaker_id"), word["start"], word["end"], word["text"])
```

To check two speakers, replace the file name with a conversation recording. `diarize="true"` enables speaker labels, and `speaker_id` lets you read them from the result. The TXT file stores just the text here; the full response with timings and speakers stays in the JSON. Provider's specification and example: STT API, STT quickstart.

A2. TTS — text → MP3

Run it as a separate example. In the `voice_id` field enter the identifier of a voice available via the API on your account. To use a clone, provide its identifier.

```
from getpass import getpass
from pathlib import Path

import requests

api_key = getpass("Paste your ElevenLabs API key (hidden): ").strip()
voice_id = input("Paste the voice_id of the chosen voice: ").strip()

response = requests.post(
    f"https://api.elevenlabs.io/v1/text-to-speech/{voice_id}",
    headers={"xi-api-key": api_key},
    params={"output_format": "mp3_44100_128"},
    json={
        "text": "Hi! This voice was generated from text.",
        "model_id": "eleven_multilingual_v2",
    },
    timeout=120,
)

response.raise_for_status()
Path("result.mp3").write_bytes(response.content)
print("Saved result.mp3 — download the file from the Colab file panel.")
```

`raise_for_status()` stops the example on an HTTP error, before the response is saved as a result. TTS parameters: Create speech.

Materials: BrainHack2026_ASR_Workshop repo · AIMed AGH